

Content-Addressed Continual Memory with On-Demand Capacity

Xinyu Yang
pana.yang@hotmail.com
ORCID: 0009-0007-2600-0948

August 29, 2026

Abstract

We study a continual-learning memory in which storage is addressed by content, capacity is allocated on demand from the stream without task identifiers, and new capacity is appended without perturbing anything already stored. On a relational stream whose regimes are distinguishable from their inputs, the mechanism retains 49.5% of probed facts zero-shot against 15.1% for a monolithic sparse-coded control of the same width, which is itself at the level of a linear baseline; the advantage narrows from 34 to 25 points between eight and thirty-two regimes without closing. Sharing the payload transform across regimes while keeping per-regime readouts adds 16.4 points over a fully private variant (six seeds, $p < 0.001$). Allocating readout rows only for targets a regime actually emits reduces occupied capacity by $7.6\times$ at no cost in accuracy, and appending capacity leaves existing weights byte-identical and existing predictions bit-identical.

We give equal space to what the mechanism does not do, and hold those readings open rather than assert them. It carries nothing between facts beyond a single-timescale context, and retention falls as a cue is separated from its query; multi-hop routing occupies the same capacity for less accuracy, and letting the address compose as well as the value costs more still; retention declines as regimes accumulate, with capacity starvation, worsening misrouting and failing separation each excluded by measurement, leaving retrieval difficulty as the surviving account. We report these as measurements whose interpretation is unsettled because, in the course of this study, six measurements that read as findings turned out to have been produced by the measurement rather than the mechanism.

1 Introduction

A learner that must absorb an unbounded stream with bounded parameters has to discard something. Regularisation methods such as elastic weight consolidation [Kirkpatrick et al., 2017] give up plasticity, protecting old weights at the cost of new learning. Methods that simply keep training give up the past. A third option is to *allocate*: give distinct regimes distinct storage, so that degradation is uniform rather than selective. Dynamically expanding architectures [Rusu et al., 2016, Yoon et al., 2018] take this route and are routinely criticised for adding parameters per task without demonstrating that the addition is proportionate to new content rather than to task count.

This paper examines an allocation mechanism built around three commitments. Addressing is by content through a fixed random hash, so a fact reaches the same storage under every history. Regime identity is inferred online from the stream, so no task identifier, task boundary, or replay buffer is required. Capacity is appended rather than resized, so adding storage leaves what is already there untouched, which we check exactly rather than statistically.

We report what these commitments buy and what they cost. They buy a large advantage where regimes are distinguishable, a capacity footprint that follows emitted content rather than vocabulary size, and a per-token cost independent of sequence length. They cost the ability to

associate items across a gap, which content-only addressing forbids by construction, and they leave retention declining as regimes accumulate.

Contributions.

1. An allocation mechanism whose two addressing factors — content to a memory slice, inferred regime to a readout — are separately measurable, with no task supervision at any point (§3).
2. Evidence that sharing the payload transform while privatising the readout is the effective split for a stream whose regimes share surface form and differ in targets, confirmed from both directions (§5.2).
3. A capacity result: sampling negatives from a regime’s own emitted targets makes occupancy follow content, $7.6\times$ smaller at equal accuracy (§5.3).
4. A non-destructiveness property, verified exactly rather than statistically: appending capacity leaves stored weights and predictions unchanged bit for bit (§5.4).
5. Measurements of what the mechanism does not currently do — cross-gap association, multi-hop addressing, and retention as regimes accumulate — reported with the alternative explanations we could exclude, and left open where we could not (§6).

2 Related work

Continual learning. Catastrophic interference in distributed representations [McCloskey and Cohen, 1989, French, 1999] motivates three broad families [Parisi et al., 2019, De Lange et al., 2021]: regularisation [Kirkpatrick et al., 2017], rehearsal, and architectural expansion [Rusu et al., 2016, Yoon et al., 2018]. Our mechanism is architectural, but differs from prior expansion methods in that it requires no task boundary to decide when to expand; the decision is made from the dispersion of an inferred regime’s own statistics. Task-free continual learning [Aljundi et al., 2019] shares this aim.

Conditional computation. Structurally the mechanism is a single-layer, top-1 sparsely-gated mixture of experts [Shazeer et al., 2017, Fedus et al., 2022] whose router is a fixed random hash rather than a learned gate, and whose expert count grows. We treat the unlearned router as a limitation to be reported rather than a feature: it forgoes load balancing and learned specialisation, and it is precisely what prevents context from influencing retrieval (§6).

Addressable and distributed memory. Content-addressed storage with sparse codes follows sparse distributed memory [Kanerva, 1988] and hyperdimensional computing [Kanerva, 2009]. Recoverable association of *pairs*, which a smoothed context summary cannot provide, is available through holographic reduced representations [Plate, 1995] or fast-weight outer products [Ba et al., 2016, Schlag et al., 2021]; we identify this as the missing ingredient rather than claim it.

Consolidation across timescales. Cascades of variables at multiple timescales [Fusi et al., 2005, Benna and Fusi, 2016] target the capacity–lifetime tradeoff of bounded synapses. We tested such a cascade in two roles, as weight consolidation and as input context, and report both outcomes.

Attention and its costs. Self-attention [Vaswani et al., 2017] pays $O(N)$ per token in context access and degrades on material in the middle of long contexts [Liu et al., 2024]. A content-addressed memory pays a cost independent of N ; we quantify the crossover in §5.5.

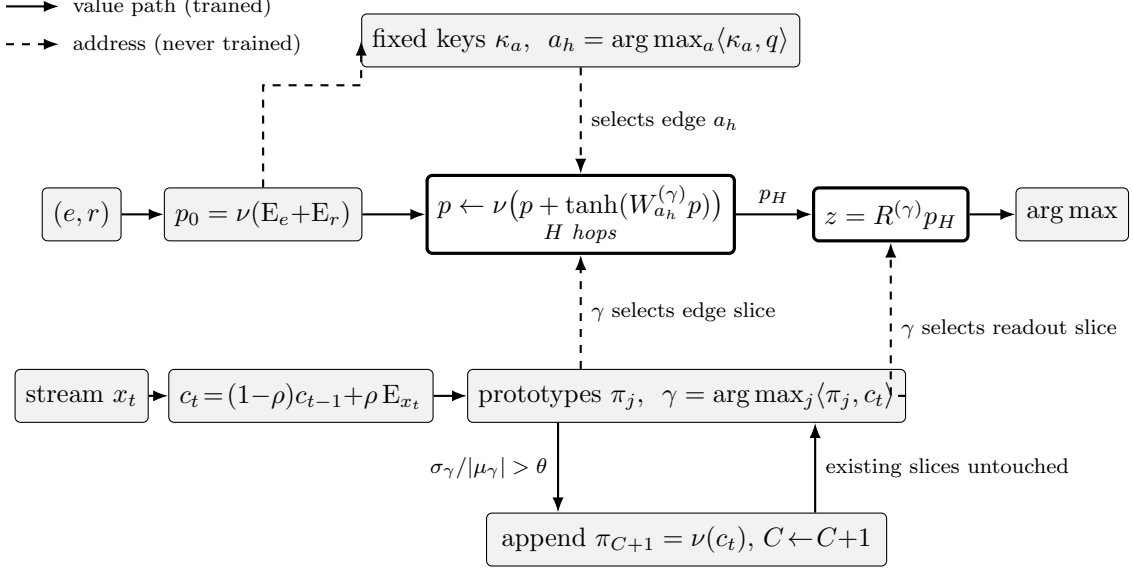


Figure 1: The mechanism. Two addressing factors, neither of them learned, decide *where* an observation is stored; the two heavy-outlined blocks are the only ones holding trained parameters. Content selects the edge, so a fact reaches the same transform under every history (§3.2); the regime inferred from the stream selects the readout slice, and the edge slice too unless the transform is shared, so facts that disagree across regimes never share a row. Capacity is appended when a class’s own similarity distribution disperses, which by §3.6 leaves every existing offset and value untouched.

Vocabulary growth. That distinct types grow sublinearly in a Zipf-distributed stream is Heaps’ law [Heaps, 1978]; we use it to ask whether allocated capacity tracks content, and report that in our generator it does not, for reasons traced to the generator itself (§6).

3 Method

3.1 Notation

Let $\mathcal{V}$ be the vocabulary, $V = |\mathcal{V}|$, and d the code width. The embedding table $E \in \mathbb{R}^{V \times d}$ has rows drawn independently and uniformly from the unit sphere S^{d-1} at construction and is *never* updated; E_x denotes the row of token x . Write $\nu(x) = x/\|x\|_2$ for ℓ_2 normalisation and δ_t for the indicator vector of token t .

Storage lives on a small-world ring $G = (\mathcal{N}, \mathcal{E})$ with $|\mathcal{N}| = n$ nodes: node u has out-neighbours consisting of its two ring neighbours plus a fixed number of random shortcuts, giving out-degree D_u and $|\mathcal{E}| = M = \sum_u D_u$ directed edges. Every edge $a \in \mathcal{E}$ carries two objects: a *key* $\kappa_a \in \mathbb{R}^d$, drawn once at construction and never updated, and, for each regime class c , a learned matrix $W_a^{(c)} \in \mathbb{R}^{d \times d}$. Each class c additionally owns a readout $R^{(c)} \in \mathbb{R}^{V \times d}$ and a prototype $\pi_c \in S^{d-1}$. The number of live classes C is not fixed in advance; it grows (§3.5).

A fact is a triple $(e, r, t) \in \mathcal{V}^3$: entity, relation, target. The stream is a sequence of tokens $x_1, x_2, \dots$ from which facts are read off, seen once each in the order they arrive.

3.2 Two addressing factors

Content to edge. The payload and the address are computed from the same tokens but kept separate, so that the transform and readout always act on content:

$$p_0 = \nu(E_e + E_r), \quad q = \nu\left(p_0 + \frac{g}{\sqrt{m}} P \xi_t\right), \quad (1)$$

where $\xi_t \in \mathbb{R}^m$ is a concatenation of context traces at several timescales, P a fixed random projection and g a gain. The main configuration uses $g = 0$, in which case $q = p_0$ and the address depends on the fact alone; $g > 0$ is the variant of §6. Entry and each hop are argmaxes over fixed keys,

$$u_0 = \arg \max_{u \in \mathcal{N}} \langle \kappa_{(u,1)}, q \rangle, \quad a_h = \arg \max_{a \in \text{out}(u_h)} \langle \kappa_a, q \rangle, \quad u_{h+1} = \text{head}(a_h), \quad (2)$$

for $h = 0, \dots, H-1$. Since κ is fixed and, at $g = 0$, q is a deterministic function of (e, r) , the walked path $(a_0, \dots, a_{H-1})$ is a deterministic function of the fact. Two consequences follow without measurement: routing consistency — the fraction of probed facts that return to the edges their writes went to — is exactly 1, and no amount of subsequent learning can move a stored fact’s address. We report 1.000 in the tables as a check on the implementation, not as a result.

Regime to readout. A single-timescale exponential average of the stream, with rate ρ , is matched against the prototypes:

$$c_t = c_{t-1} + \rho (E_{x_t} - c_{t-1}), \quad \gamma_t = \arg \max_{j \leq C} \langle \pi_j, c_t \rangle. \quad (3)$$

No task identifier enters anywhere: γ is inferred from the stream, and the mechanism is never told which regime is live, nor when one ends. The prototypes receive no gradient; they are *placed* (§3.5), not trained.

3.3 Memory and forward pass

The value path applies the selected edge transform residually,

$$p_{h+1} = \nu \left(p_h + \tanh(W_{a_h}^{(\gamma)} p_h) \right), \quad h = 0, \dots, H-1, \quad (4)$$

with $\tanh$ elementwise, and the class readout produces logits $z = R^{(\gamma)} p_H \in \mathbb{R}^V$, $\hat{y} = \text{softmax}(z)$.

The split between what is shared and what is private follows the generative structure rather than preference. A payload is built from an entity and a relation, which are common across regimes, so W can be shared ($W^{(\gamma)} \equiv W$); targets are exactly what regimes disagree about, so $R^{(\gamma)}$ must not be. Section 5.2 confirms this from both directions — by sharing the transform and by sharing the readout.

3.4 Learning

Training is online, single-pass and prequential: for each fact the model predicts, is charged the loss, and only then writes. There is no replay buffer, no task boundary, no epoch and no second pass.

The readout takes a delta-rule step restricted to a *touched set*

$$T = \{t\} \cup S, \quad S \sim \text{Unif}^k(\mathcal{T}_\gamma), \quad R_v^{(\gamma)} += \eta (\mathbb{1}[v=t] - \hat{y}_v) p_H^\top \quad \text{for } v \in T, \quad (5)$$

where $\mathcal{T}_\gamma$ is the set of targets class γ has already emitted and k the number of sampled negatives ($k = 0$ recovers the dense update over all V rows). Rows outside T are left *exactly* zero rather than nearly zero, which is what makes occupied capacity a measure of stored content (§5.3).

The error is then backpropagated along the path that was actually walked, and along no other. With $\nabla_{p_H} L = (R^{(\gamma)})^\top (\hat{y} - \delta_t)$, each hop is differentiated exactly through the normalisation and the residual $\tanh$ block of (4): writing $b_h = W_{a_h}^{(\gamma)} p_h$, $u_h = p_h + \tanh(b_h)$ and $\lambda_h = \|u_h\|$,

$$\nabla_{u_h} L = \frac{1}{\lambda_h} \left(I - p_{h+1} p_{h+1}^\top \right) \nabla_{p_{h+1}} L, \quad \nabla_{b_h} L = (1 - \tanh^2(b_h)) \odot \nabla_{u_h} L, \quad (6)$$

$$W_{a_h}^{(\gamma)} = \eta \nabla_{b_h} L p_h^\top, \quad \nabla_{p_h} L = \nabla_{u_h} L + (W_{a_h}^{(\gamma)})^\top \nabla_{b_h} L. \quad (7)$$

The gradient reaches neither the keys κ , nor the prototypes π , nor the embedding table: every quantity that decides *where* an observation goes is outside the computational graph. Decay is applied lazily, by carrying a per-slice scale and materialising a slice only when it is touched, which is algebraically exact rather than an approximation.

3.5 Allocation

Let $\text{sim}_t = \max_j \langle \pi_j, c_t \rangle$ and let $\mu_\gamma, \sigma_\gamma^2$ be running estimates of its mean and variance conditioned on the class γ that won. A class is split when its own similarity distribution is too dispersed for τ consecutive steps,

$$\frac{\sigma_\gamma}{\max(|\mu_\gamma|, \epsilon)} > \theta, \quad \text{whereupon} \quad \pi_{C+1} \leftarrow \nu(c_t), \quad C \leftarrow C + 1. \quad (8)$$

Dispersion is used rather than an outlier test because outlier tests fail in two ways we measured. Against a *global* running distribution of sim they fail once regimes are numerous, since that distribution becomes their mixture and nothing is unusual relative to it, and every regime collapses onto one class. Against a class’s *own* distribution an outlier test fails too, because a class absorbing more regimes raises its own variance, so a threshold of the form $\mu_\gamma - \theta \sigma_\gamma$ falls exactly when splitting becomes more necessary. Dispersion is scale-free in both senses: a class serving one regime has tightly clustered similarities whatever the value of C .

The new prototype is placed *at* the current context direction. Placing it on the residual orthogonal to the existing prototypes — which an earlier version did — fails for a geometric reason: in high dimension the orthogonal residual is nearly orthogonal to the data as well, so the new prototype wins no argmax at all, so classes are allocated while every regime still routes to one.

3.6 Expansion is exact

Both R and W are stored class-major, so class c occupies a contiguous block at offset $c \cdot Vd$ (respectively $c \cdot Md^2$). Appending class $C + 1$ therefore changes no offset of any class $j \leq C$ and writes no byte inside one. Consequently, for any input whose inferred regime satisfies $\gamma \leq C$, the logits $z = R^{(\gamma)} p_H$ are *bit-identical* before and after expansion, since the arithmetic performed is the same on the same operands in the same order. This is a property of the layout, and we assert it as a unit test on weights and logits rather than estimating it statistically.

3.7 Cost

Per observed token, addressing costs $O((n + H\bar{D})d)$ for the argmaxes of (2) and $O(Cd)$ for (3); the memory costs $O(Hd^2)$ for (4); the readout costs $O(Vd)$ to score and $O(|T|d)$ to write. None of these depends on how many tokens preceded, which is the property of interest: dot-product attention over a context of N tokens costs $O(Nd)$ per token for the same function, and the two cross at $N = d$. The readout term is the one that does not yet follow occupancy, and it is what §8 proposes to make sparse.

4 Experimental setup

The source is a multi-domain relational graph walked continuously, with facts stratified by visit frequency into hub, mid and tail tiers under a Zipf distribution. Two modes are used. In **Mode A** each regime owns its entities, so regime identity is present in the input. In **Mode B** entities and relations are byte-identical across regimes and only targets differ, which removes the routing

signal and, by construction, any shared target to amortise; we use it as an adversarial bound rather than as the main setting.

Evaluation is zero-shot retention of probed facts under a frozen memory, with the probe restoring the context it needs and returning the state it displaced. All figures in this paper are extracted by a single parser that anchors on the results table header and reads the column by name.

5 Results

5.1 Where regimes are distinguishable

regimes	this work	monolithic	advantage
8	49.53%	15.10%	+34.43pp
16	43.07%	12.03%	+31.04pp
32	37.65%	12.20%	+25.45pp

Table 1: Mode A; three seeds at eight and sixteen regimes, one at thirty-two. The monolithic control sits at the level of a linear baseline without sparse coding (15.30%), so the task is not solvable by predicting frequent targets. The advantage narrows as regimes accumulate but does not close. We did not obtain a monolithic baseline at sixty-four regimes; a single cell costs about nine hours here, and the run was lost.

5.2 Sharing the transform, privatising the readout

Mode B, twelve regimes, six seeds	zero-shot	paired difference
Shared transform, private readouts	29.42%	+16.43pp, 6/6, $p < 0.001$
Fully private	12.98%	—
Monolithic	21.28%	+8.13pp, 5/6, $p = 0.119$

Table 2: Sharing the transform is established against the private architecture. The advantage over the monolithic control is not.

Sharing the *readout* as well costs 27.50 points. A common readout block is updated on every write and therefore accumulates every regime’s target mapping, while Mode B is defined by the same (e, r) pointing at different targets per regime; it averages contradictions. The asymmetry of §3 is thus confirmed from both sides, one by making the change and the other by testing its opposite.

5.3 Capacity follows content

	occupied rows	rows per class	accuracy
Dense delta rule	45056/45056 (100%)	4096	32.6%
Sampled from emitted targets	5918/45056 (13.1%)	538	33.8%

Table 3: A $7.6\times$ reduction in occupied capacity at no cost in accuracy.

5.4 Expansion does not disturb what is stored

Appending a class slice leaves every existing weight byte-identical and **forward** bit-identical, asserted as a test rather than measured statistically. A monolithic model cannot offer this: its capacity is the sparse-coding width, and widening it changes the projection through which every stored code was written.

5.5 Cost per token is independent of context length

Context access costs Hd^2 per token regardless of sequence length, against Nd for attention; the crossover is at $N = d$ and the ratio is $128\times$ at $N = 4096$. Against ourselves: overall activation is 6.178% of memory parameters, of which the edge memory contributes 1024 and the remainder is the readout scoring all V rows. Sparsity currently holds for the expert part only.

6 Open questions

Every negative reading in this study that we examined closely turned out, at least once, to be produced by the measurement rather than by the mechanism (Appendix A). We therefore report the following as measurements whose interpretation is unsettled, rather than as properties of the architecture.

Can the mechanism associate across a gap? Nothing is carried between facts beyond a single-timescale context, so a cue survives $(1 - \rho)^{\text{gap}}$.

gap	cue remaining	accuracy
1	0.990	10.10%
16	0.851	10.33%
64	0.526	8.40%
256	0.076	6.83%

Table 4: Paired degradation +3.27pp, 5/6 seeds, $p = 0.013$. The location of the cliff was predicted from the context rate before the run.

Exact association under constant memory is information-theoretically unavailable, so the design question is what a lossy code should retain. A multi-scale average answers *which regime* and cannot answer *which item*. Placing context into the address recovers roughly a tenth of the gap and costs lookup accuracy, consistent with content-only addressing being what forbids association in the first place.

The evaluation itself is unfavourable to the mechanism, and we want that on the record. Each run fixes a single gap, and a single gap is best served by whichever single time constant happens to match it; a multi-scale context can therefore show no advantage under this protocol by construction, because the comparison is decided by the evaluation rather than by the code.

Under a mixed-gap protocol that cycles horizons within a run, the sign of the effect changes: putting the multi-scale trace into the address is worth +0.67pp on average against not doing so, where under a single fixed gap the same setting bought +0.40pp at the long end while costing 0.47pp at the short one. Three seeds and two of three in the favourable direction is far too little to establish anything, but it is consistent with the protocol rather than the code having decided the earlier comparison. We regard both protocols as preliminary: a gap constructed in a synthetic stream fixes what has to be carried and for how long, and cannot show what a real dependency structure demands. Establishing this needs evaluation on natural language at scale, which we have not done.

Whether this is a limit of the mechanism or of this particular probe is not settled. Three earlier versions of the same measurement were invalid, each for a different reason, and the fourth is the one reported here; placing context into the address recovers roughly a tenth of the gap, which suggests the addressing scheme rather than the context code may be the binding constraint.

hops	zero-shot	occupied rows
1	55.15%	750
2	46.00%	737
3	29.63%	750

Table 5: Identical occupancy, monotonically worse retention. The under-training explanation is excluded: writes per edge *rise* with hops, from 3246 to 9222, because each fact touches more edges.

Does multi-hop addressing ever pay? We do not conclude that depth is unnecessary, and the setup is narrow enough that we could not. Multi-hop is the only form of depth this architecture has, and it is depth in the transform rather than in the routing: hop $h+1$ transforms hop h ’s output, but every hop selects its edge with the same fixed copy of p_0 , so the sequence of edges a fact visits is settled by its original content. An earlier version of this comparison was taken while regime separation had collapsed; the present one uses a single source, one scale, and a lookup task whose targets are determined entirely by the current fact. A source that rewards composition — where an answer follows from chaining two stored relations rather than retrieving one — is exactly where combinatorial addressing would be expected to pay, and we have not built one. What the table shows is that paths cost accuracy when nothing requires a path.

regimes	zero-shot	occupied	rows/regime	home classes per regime
8	49.53%	530	66	0.75
16	43.07%	1142	71	0.64
32	37.65%	2412	75	0.70
64	34.80%	5198	81	0.78

Table 6: Monolithic baselines are available to thirty-two regimes and are reported in §5.1; the sixty-four-regime cell was not obtained.

Why does retention decline as regimes accumulate? Three explanations are excluded by the data. Capacity is supplied linearly, so this is not starvation. Misrouted write magnitude is 57.0, 54.7, 51.0, 55.0 per cent — flat — so it is not worsening interference. Home classes per regime is flat, so separation does not degrade. What remains is retrieval: confusable stored targets grow $9.8\times$ while accuracy falls to $0.70\times$, and distinct predictions rise from 47 to 57 rather than collapsing.

These exclusions constrain the explanation without establishing it, and the monolithic baselines beyond sixteen regimes are still running. We state the retrieval account as the surviving candidate rather than as the finding.

Does directed forgetting reclaim anything? We cannot say. Every measurement we made of it is confounded. The earlier ones ran while a per-domain snapshot keyed by ground truth was reaching the training path, and the later one, which added capacity pressure so that a freed slice would be worth something, ran while the allocation criterion was collapsing every regime

into a single class — so there were no separate slices to reclaim in the first place. We report no number for it, and note only that a valid test requires contention between regimes for storage *and* a working allocator, which we did not have simultaneously.

Does acquiring a regime become cheaper as regimes accumulate? Later regimes do cost less per unit accuracy in our runs, but our measurement staggers arrival, and arrival order is itself sufficient to produce the effect: a regime arriving late enters a system whose transform and prototypes are already shaped. Separating order from content requires a design we have not run, so we leave the question open rather than answer it from a measurement that cannot distinguish the two (Appendix B).

Is the decline with scale a power law? Retention falls roughly as $N^{-\alpha}$ with $\alpha \approx 0.16$ over the measured range, N being the number of stored items. Whether this exponent is a property of the mechanism, and whether stronger addressing rather than more capacity moves it, is measurable without new machinery.

What should a constant-memory context retain? Exact association across a gap is information-theoretically unavailable under constant memory, so the design question is what to keep. A multi-scale average answers *which regime* and cannot answer *which item*. Sparse retention of salient items, or superposition with approximate unbinding [Plate, 1995, Ba et al., 2016], keeps identity where an average cannot.

Does the plasticity–stability frontier extend or shift? The frontier traced by our mechanism and that traced by the monolithic control barely overlap in plasticity, so at present only reachability can be compared, not position.

7 Discussion

7.1 Why exact context cannot be cheap

To associate any point in a stream with any other exactly, a system must be able to answer for an arbitrary pair, which requires either retaining every state and comparing against them — $O(N)$ memory and $O(N)$ work per query — or precomputing the pairs, $O(N^2)$ in total. Self-attention pays exactly this and buys exactness within its window in return [Vaswani et al., 2017]. The cost is not an implementation artefact; it is what pairwise access costs.

Under constant memory the question changes. Once the stream is longer than what the memory can hold distinctly, some pairs must become unanswerable, and the design decision is which ones. This is the same shape of decision that continual learning already forces on the weights: bounded capacity against an unbounded stream means something is given up, and methods differ in what [Kirkpatrick et al., 2017, Rusu et al., 2016]. Our mechanism extends that choice to the context.

7.2 A fuzzy background instead of a transcript

The position we take, and the one we find natural given the biological case, is that a learner does not need a transcript of its context. A slowly varying state that says *what kind of situation this is* can serve as an anchor for retrieval, leaving the identity of individual past items unrecoverable. Our context is exactly that: an exponential summary matched against prototypes, which conditions retrieval without recording what occurred.

Our measurements track this division cleanly. The mechanism answers “which regime” well — regimes separate, and retention exceeds the monolithic control by 25 to 34 points — and fails

at “which item occurred” as soon as a cue must survive a gap (§6). That is the anchor working as an anchor and failing as a record, which is what it is.

We do not claim this is the only division, or the best one. At least four alternatives are available and differ in what they keep: sparse retention of a few salient items keeps identity and discards time; superposition with approximate unbinding keeps many items at graceful degradation [Plate, 1995]; outer-product fast weights keep pairs at a capacity that scales with d^2 [Ba et al., 2016, Schlag et al., 2021]; and a hybrid in which a fuzzy anchor selects a slice while a small exact buffer covers the recent past would give up long-range exactness only. Which of these is right is an empirical question we have not settled, and the answer may well depend on what the stream is.

7.3 Where this leaves three familiar problems

Catastrophic forgetting. Allocation does not solve interference so much as convert it. Where a shared representation loses old material as new material overwrites it [McCloskey and Cohen, 1989], storage that is separated by regime removes that channel and substitutes another: retrieval among a growing inventory. The exchange rate looks favourable in our measurements — confusable stored targets grow $9.8\times$ while retention falls to $0.70\times$, and the distinct predictions made rise rather than collapse — but it is a substitution, not a removal, and the cost reappears as scale.

The stronger reason to prefer it is not the rate but the shape of the two failures. Interference in a shared representation is destructive and open-ended: material that is overwritten is gone, and the only remedies are more capacity or less learning. Retrieval difficulty is non-destructive and sub-proportional: the material is still stored, merely harder to find, and a tenfold larger inventory cost us thirty per cent of retention. More importantly the two differ in what can be done about them. Interference is answered only by spending parameters; retrieval difficulty is answered by addressing better, which is a design axis rather than a budget.

That said, an exponent of roughly 0.16 compounds, and a mechanism whose retention decays with everything it has ever learned would not scale. Several remedies follow directly from the diagnosis, and we list them in order of how directly they attack the measured quantity, which is the number of candidates a retrieval must discriminate among.

1. *Bound the scored set.* Inference currently scores all V rows of the live slice while a class holds 39–41 occupied rows. Scoring only occupied rows makes the candidate count follow content directly, which is the quantity in the exponent (§8).
2. *Shard the memory.* A coarse router over several independent memories, each with its own allocator, keeps any one inventory bounded while total capacity grows with the number of shards — a mixture of experts whose experts are memories rather than layers [Shazeer et al., 2017, Fedus et al., 2022]. Growth then adds shards rather than enlarging a single candidate set, so the exponent becomes a property of shard size instead of a property of everything ever learned. The cost is a second routing decision that can be wrong, and the open question is whether material that belongs to two shards must be duplicated.
3. *Address hierarchically.* Within a shard, a coarse code selecting a slice and a fine code over that slice’s own targets makes discrimination logarithmic in the inventory rather than linear, at the cost of a coarse-level error being unrecoverable at the fine level (§8).
4. *Grow the code with the inventory.* Discriminating N items needs a code whose capacity keeps pace; letting d grow like $\log N$ would hold the exponent at the price of memory per row and of the byte-identical expansion property, since widening the code changes existing rows.

5. *Merge redundant slices.* If allocation over-splits, inventory reflects duplicates rather than distinct content. Consolidating slices whose similarity profiles coincide would keep the inventory a measure of what is actually distinct.
6. *Sample negatives adversarially.* Discrimination is currently trained against negatives drawn from a regime’s own emitted targets. Drawing them from the current nearest confusables instead spends the code where collisions actually occur.

The first two are the ones we would try first, and the second is the one that changes the asymptote rather than the constant.

Lost in the middle. Attention degrades on material by its position in the context [Liu et al., 2024]. Content addressing has no position term at all: a fact stored early is reached by the same hash as one stored late, so retrieval carries no positional decay. What decays instead is regime identification, because the context summary is recency-weighted, and what degrades with load is retrieval, because the inventory grows. The degradation is therefore moved off position and onto inventory and recency, which is a different failure surface rather than a smaller one — a prediction that is testable directly, and that we have not tested.

Plasticity and stability. We do not beat the frontier. Our mechanism and the monolithic control barely overlap in plasticity: the control saturates near 0.31 while ours spans 0.32 to 0.92, so at present the honest statement is about reachability, not position. Whether the frontier is extended or merely shifted requires a regime where both operate, which we did not construct.

7.4 What this is not for, and what it might be for

A statically trained model, evaluated as a static tool, remains the accuracy ceiling. It is trained to convergence over a fixed corpus with many passes; a single-pass online memory will not match it on a benchmark drawn from that same corpus, and we do not suggest otherwise. For answering ordinary questions from knowledge fixed at training time, the static model is the right instrument and this one is not.

The case for a mechanism like ours is different in kind: acceptable accuracy combined with the ability to keep working. We would also note that accuracy on a fixed source is a narrow proxy. It measures what a model knows at the moment it was frozen, not what it can absorb afterwards, not what it costs to keep running, and not what happens after weeks of operation — none of which a static benchmark is constructed to reveal. The properties measured here point at settings where those are the binding constraints.

Sessions measured in weeks. Per-token cost does not grow with context and there is no cache to retain, so an agent can run for as long as the task lasts rather than as long as a window allows. What it absorbed on the first day is stored rather than scrolled out.

Embodied and IoT settings. A robot or sensor node meets regimes that were not enumerated in advance and receives them once, in order, without labels. Allocation from the stream and single-pass writing match that shape; retraining cycles do not.

Edge and green deployment. Context access is Hd^2 per token independently of history, and a write touches one slice and a handful of rows. The cost profile is close to flat in the length of the deployment, which is the quantity that dominates energy over a long-running installation.

Absorbing new domains after deployment. Capacity appends without disturbing what is stored, so a fielded model can take on a new domain without a retraining cycle. The exactness matters more than the magnitude: one can state that adding capacity did not change existing behaviour, rather than measure that it changed little.

Targeted deletion, and interpretability more broadly. A regime occupies identifiable slices, so removing one is a local operation rather than a retraining problem — the natural test of

the same structure that makes expansion non-destructive, and one we have not run. The same property supports weaker but broader questions: which slice serves this material, what did this write change, what would removing this regime cost. A learned router gives no comparable account of itself, and a dense model gives none at all. We think this is the more interesting direction of the two, since deletion is one query among many that an addressable, inspectable store can answer.

Long-tail material. Capacity follows emitted content rather than vocabulary size, so rare items occupy their own rows instead of being averaged into frequent neighbours.

7.5 Fabrication

We did not measure hallucination and make no claim about rates. But three structural properties bear on it and are worth stating, since each is a consequence of the architecture rather than of tuning.

Training and inference are not separate modes: the same forward path writes and reads, and what a query returns is what some slice holds. There is no generation stage distinct from retrieval in which content can be composed that was never stored.

Regimes occupy separate readout slices, so an answer is produced from the slice the context selected. Blending material across domains — one plausible route to a confident wrong answer — is limited by the same separation that limits interference, rather than by a penalty term.

Sparse occupancy makes absence representable. A class holds tens of occupied rows out of tens of thousands, so “nothing here scores well” is a state the mechanism can be in, which a dense readout normalised over the full vocabulary cannot easily express. Whether this translates into calibrated abstention is untested and would be the obvious experiment.

7.6 On whether growing capacity is disqualifying

Expansion invites the objection that a method which adds parameters has not solved anything. We think the objection is right about unbounded growth and wrong as a general principle, and the biological case is instructive on the distinction. A brain does not hold its knowledge in a fixed parameter vector: connectivity is combinatorial, signalling is graded rather than discrete, and the number of distinguishable synaptic states is correspondingly large. A system with that much effective capacity can afford to allocate in proportion to content, because content is small relative to what it can hold.

Digital parameters are finite and discrete, so the same strategy is only tenable if per-shard inventory is bounded — which is the argument for sharding above, and the reason we regard unbounded growth in a single memory as a defect to be fixed rather than a property to be defended. What the biological case does suggest is that allocation proportional to content is not disqualifying in itself. What must be bounded is not the total, but the number of things any single retrieval has to discriminate among.

8 Architectural variants worth trying

Each variant below targets a specific measurement in this paper rather than a general intuition.

Bound scoring to the emitted rows. Inference scores all V rows of the live readout slice, while a class holds 39–41 occupied rows against a vocabulary of 32768. Restricting the argmax to rows the class has emitted, with the remainder contributing a constant, would cut inference cost by nearly three orders of magnitude and would make the activation figure of §5.5 sparse in both halves rather than only in the expert half. Nothing about the mechanism resists this; it is unimplemented, not difficult.

Address hierarchically to bound the candidate set. The decline with scale tracks the number of confusable stored items rather than capacity, misrouting or separation (§6). If that account is right, the fix is not more capacity but a smaller candidate set at retrieval: a coarse regime code selecting a slice, then a within-slice code over that regime’s own targets, so that discrimination is against tens of alternatives rather than thousands. This predicts that the exponent α should fall with the depth of the addressing hierarchy, which is directly testable.

Add a term that stores pairs. A smoothed context can report which regime is active and cannot report which item occurred, so no amount of extra timescales will support association across a gap. An outer-product term, $M \leftarrow \lambda M + p_t \otimes p_{t-\Delta}$ retrieved as Mp , stores pairs recoverably at a capacity that scales with d^2 [Ba et al., 2016, Schlag et al., 2021], and is the same algebraic object as the edge transform already present. The open question is whether it can sit alongside content addressing without dissolving the stability that content addressing provides.

Give the router a learned residual. The fixed random hash buys routing consistency of 1.000 and forgoes load balancing and learned specialisation. A learned correction added to the key, small enough that a fact’s destination remains stable within a regime, would recover part of what a gated mixture of experts gets from its router [Shazeer et al., 2017] without giving up the property that makes expansion non-destructive.

Let the address compose, not only the value. The value path already composes: hop $h + 1$ transforms hop h ’s output and gradients flow along the walked chain. What repeats is the *address* — every hop selects its edge with the same fixed copy of p_0 , so the sequence of edges a fact visits is settled before the walk begins.

We implemented the obvious remedy, copying the transformed payload into the routing key after each hop so that hop $h + 1$ selects from hop h ’s output, and it makes matters worse. Within its own configuration, which uses a larger ring than the multi-hop table above and is therefore not directly comparable to it, accuracy falls from 51.43% to 41.03% at two hops and from 35.10% to 22.83% at three, so the gap to a single hop widens rather than closes. The reason is visible in the same run. Routing consistency, which the fixed key guarantees at 1.000, falls to 0.910; a fact’s path now depends on weights that move, so nine per cent of probed facts no longer reach where training left them, and that loss outweighs whatever composition buys. Whether a source that genuinely requires chaining two stored relations would reverse the balance is untested — on a lookup whose target is fixed by the current fact there is nothing for a composed path to find — so this settles the version we tried rather than the idea.

A note on tokenisation. We recommend against subword tokenisation for this architecture. Byte-pair encoding is designed to flatten the frequency distribution, and this mechanism depends on that distribution in two places: allocation splits a class when its similarity dispersion rises, which requires regimes to differ in what they emit, and capacity follows the count of distinct emitted targets, which requires a target to be a unit rather than a sequence. A rare entity split into several frequent subwords loses its identity twice over — its payload becomes a sum of common pieces, which collides in the content hash with every other entity sharing those pieces, and its target becomes a sequence with no single row to occupy. The long tail is exactly where a continual learner has something to gain, and subword vocabularies are built to spend it. A word- or entity-level vocabulary, or any tokenisation that keeps rare units intact, preserves both the Zipf structure the allocator reads and the one-target-one-row correspondence the capacity argument relies on.

9 Limitations

Results are on a synthetic relational source; no natural-language corpus is evaluated, and a stream we constructed fixes what has to be learned and what can be shared, which is precisely what a real corpus would decide for us.

The monolithic control is established at eight, sixteen and thirty-two regimes, with one seed at thirty-two and none at sixty-four. It is not parameter-matched in either direction: at eight regimes it holds 16.8M readout weights against 11.6M allocated by our mechanism, of which about 17K are occupied. The comparison is therefore between two models of comparable width rather than of equal size, and the occupancy figures should be read as describing our mechanism rather than as a like-for-like efficiency claim.

Cost comparisons between the dense and sampled write paths are not commensurable, because write magnitude accumulates over a different number of rows in each. The private and shared arms likewise differ in parameter count, which strengthens the accuracy comparison between them but confounds any comparison of their costs.

Sample sizes are small throughout: six seeds for the sharing result, three for most others, one for the largest control. Several effects in this study weakened or reversed between three seeds and six, so single-digit seed counts should be read accordingly.

Whether acquiring a regime becomes cheaper as regimes accumulate is not settled here: our measurement staggers arrival, and arrival order alone can produce the effect (Appendix B).

10 Conclusion

Content addressing with on-demand allocation buys a large advantage where regimes are distinguishable, a capacity footprint that follows emitted content, and a per-token cost independent of context length, without disturbing what is already stored. The same commitment that makes storage stable — addressing by content alone — is what prevents association across a gap, and the cost of that trade is measured here rather than asserted.

A Measurements invalidated during this study

Six measurements were invalidated after collection and re-run. Three were probe-side leaks in which a per-domain snapshot keyed by ground truth handed the mechanism the regime identity it was meant to infer. One was the allocation criterion of §3.5 in an earlier form, which compared novelty against a global running distribution and collapsed every regime into a single class at scale, voiding the savings, capacity and multi-hop measurements taken under it. One was a parser that scanned past the results table into a table with the same column count and reported 54.2% as 100.00%.

Each was caught by a contradiction between statistics rather than by reading code: a count that disagreed with another count, a diagnostic that printed nothing, an accuracy that could not coexist with its own exposure figures. The instruments that survived — write locality by time since regime change, occupancy against emitted content, allocation timing against regime boundaries — are reported alongside the results because they are what made the failures visible.

B Arrival order and the cost of a late regime

Measuring whether a later regime is cheaper to acquire requires staggered arrival, and staggering introduces a confound we record here for the benefit of anyone running the same measurement.

Mode B’s targets are disjoint across regimes by construction, so nothing there can be amortised, yet the ratio is lower than in Mode A. Removing the staggering removes the effect.

	shared transform	private
Mode A, staggered arrival	0.754	0.820
Mode B, disjoint targets, staggered arrival	0.442	0.505
Mode A, no staggering (index only)	1.199	1.207

Table 7: Cost of the late regimes relative to the early ones; below one means a late regime is cheaper. Three seeds per control.

A regime arriving late therefore benefits from entering an already-shaped system, independently of whether it shares content with its predecessors, and a ratio below one cannot on its own be read as amortisation.

References

- R. Aljundi, K. Kelchtermans, and T. Tuytelaars. Task-free continual learning. In *CVPR*, 2019.
- J. Ba, G. Hinton, V. Mnih, J. Z. Leibo, and C. Ionescu. Using fast weights to attend to the recent past. In *NeurIPS*, 2016.
- M. K. Benna and S. Fusi. Computational principles of synaptic memory consolidation. *Nature Neuroscience*, 2016.
- M. De Lange, R. Aljundi, M. Masana, S. Parisot, X. Jia, A. Leonardis, G. Slabaugh, and T. Tuytelaars. A continual learning survey: Defying forgetting in classification tasks. *IEEE TPAMI*, 2021.
- W. Fedus, B. Zoph, and N. Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *JMLR*, 2022.
- R. M. French. Catastrophic forgetting in connectionist networks. *Trends in Cognitive Sciences*, 1999.
- S. Fusi, P. J. Drew, and L. F. Abbott. Cascade models of synaptically stored memories. *Neuron*, 2005.
- H. S. Heaps. *Information Retrieval: Computational and Theoretical Aspects*. Academic Press, 1978.
- P. Kanerva. *Sparse Distributed Memory*. MIT Press, 1988.
- P. Kanerva. Hyperdimensional computing: An introduction to computing in distributed representation with high-dimensional random vectors. *Cognitive Computation*, 2009.
- J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *PNAS*, 2017.
- N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang. Lost in the middle: How language models use long contexts. *TACL*, 2024.
- M. McCloskey and N. J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. *Psychology of Learning and Motivation*, 1989.
- G. I. Parisi, R. Kemker, J. L. Part, C. Kanan, and S. Wermter. Continual lifelong learning with neural networks: A review. *Neural Networks*, 2019.

- T. A. Plate. Holographic reduced representations. *IEEE Transactions on Neural Networks*, 1995.
- A. A. Rusu, N. C. Rabinowitz, G. Desjardins, H. Soyer, J. Kirkpatrick, K. Kavukcuoglu, R. Pascanu, and R. Hadsell. Progressive neural networks. *arXiv:1606.04671*, 2016.
- I. Schlag, K. Irie, and J. Schmidhuber. Linear transformers are secretly fast weight programmers. In *ICML*, 2021.
- N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *ICLR*, 2017.
- A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In *NeurIPS*, 2017.
- J. Yoon, E. Yang, J. Lee, and S. J. Hwang. Lifelong learning with dynamically expandable networks. In *ICLR*, 2018.